TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

TÀI LIỆU THỰC HÀNH

Học máy ứng dụng

Bài 8: Phân cụm K-Means

Biên soạn ThS. Nguyễn Thái Anh

Năm học 2026 – 2027

Mục lục

1 Mục tiêu bài thực hành **2**
1.1 Chuẩn bị . . . 2

2 Tự viết K-Means **2**
2.1 Nhắc lại thuật toán . . . 2
2.2 Đọc kết quả . . . 4

3 Phân khúc khách hàng **5**
3.1 Đọc kết quả . . . 6
3.2 Xếp khách mới vào cụm . . . 7

4 Chọn số cụm K **7**
4.1 Đọc kết quả . . . 9

5 Ba cái bẫy của K-Means **9**
5.1 Bẫy thứ nhất: quên chuẩn hoá . . . 11
5.2 Bẫy thứ hai: khởi tạo xấu . . . 11
5.3 Bẫy thứ ba: cụm không tròn . . . 11

6 Nén màu ảnh **12**
6.1 Đọc kết quả . . . 13

7 Bài tập **13**

8 Phụ lục A, bảng tra nhanh **15**

9 Phụ lục B, lỗi thường gặp **16**

1. Mục tiêu bài thực hành

Bảy buổi trước, dữ liệu luôn có một cột nhãn để mô hình học theo. Buổi này lần đầu tiên dữ liệu **không có nhãn**. Ta để máy tự chia dữ liệu thành các nhóm giống nhau bằng thuật toán K-Means.

Sau khi hoàn thành bài này, sinh viên có khả năng:

1. Tự viết K-Means bằng numpy và chạy từng vòng như trên slide.
2. Dùng `KMeans` của scikit-learn để phân khúc khách hàng, đọc tâm cụm theo đơn vị gốc.
3. Chọn số cụm K bằng phương pháp khuỷu tay và chỉ số silhouette.
4. Nhận ra ba cái bẫy: quên chuẩn hoá, khởi tạo xấu, cụm không tròn.
5. Dùng K-Means để nén màu một bức ảnh.

Ba bộ dữ liệu

Tệp `data/khach_hang.csv` gồm 210 khách hàng của một trung tâm thương mại, là số liệu mô phỏng dùng cho mục đích giảng dạy. Mục 5 dùng lại `data/hai_vang_trang.csv` của bài 4. Mục 6 dùng một bức ảnh có sẵn trong scikit-learn, nạp bằng `load_sample_image`, không cần tải gì thêm.

Bảng 1: *Các cột của tệp `khach_hang.csv`. Không có cột nhãn.*

Cột	Ý nghĩa
`tuoi`	Tuổi của khách.
`thu_nhap`	Thu nhập, triệu đồng mỗi tháng.
`diem_chi_tieu`	Điểm chi tiêu từ 1 tới 100, trung tâm tự chấm theo lịch sử mua sắm.

1.1. Chuẩn bị

Vẫn môi trường của bài 1, không cài thêm gì.

```
cd C:\MayHoc
hocmay\Scripts\activate
cd bai08_kmeans
```

Bảng 2: *Năm tệp mã của buổi thực hành, chạy theo đúng thứ tự này.*

Mục	Tệp	Nội dung
2	`k1_tu_viet.py`	Tự viết K-Means bằng numpy
3	`k2_khach_hang.py`	Phân khúc khách hàng
4	`k3_chon_k.py`	Chọn số cụm K
5	`k4_ba_cai_bay.py`	Ba cái bẫy của K-Means
6	`k5_nen_anh.py`	Nén màu ảnh

2. Tự viết K-Means

2.1. Nhắc lại thuật toán

1. Chọn K điểm làm tâm ban đầu.

2. **Gán**: mỗi điểm về cụm của tâm gần nó nhất.
3. **Dời tâm**: mỗi tâm dời về trung bình các điểm trong cụm của nó.
4. Lặp lại bước 2 và bước 3 tới khi tâm không đổi nữa.

Inertia

K-Means làm cho tổng bình phương khoảng cách từ mỗi điểm tới tâm cụm của nó nhỏ dần.

$$\text{Inertia} = \sum_i \|\boldsymbol{x}_i - \text{tâm của cụm chứa } \boldsymbol{x}_i\|^2$$

Cụm càng gọn thì inertia càng nhỏ.

Tệp mã dưới đây viết cả hai bước bằng numpy, chạy trên bảy điểm A tới G của slide, với hai tâm ban đầu cố ý đặt tệ là A và B.

```python
# -*- coding: utf-8 -*-
"""Bước 1: tự viết K-Means bằng numpy, chạy trên bảy điểm của slide.

Chú thích viết tiếng Việt có dấu, phần in ra màn hình viết không dấu.
"""

import numpy as np
from sklearn.cluster import KMeans

P = np.array([[1.0, 1.0], [1.5, 2.0], [3.0, 4.0], [5.0, 7.0],
              [3.5, 5.0], [4.5, 5.0], [3.5, 4.5]])
TEN = ["A", "B", "C", "D", "E", "F", "G"]


def k_means(X, tam, so_vong_toi_da=10):
    """K-Means don gian: gan diem vao tam gan nhat roi doi tam."""
    for vong in range(1, so_vong_toi_da + 1):
        # Bước gán: khoảng cách từ mỗi điểm tới mỗi tâm
        kc = np.sqrt(((X[:, None, :] - tam[None, :, :]) ** 2).sum(axis=2))
        nhan = kc.argmin(axis=1)
        print(f"Vong {vong}: tam = {np.round(tam, 2).tolist()}")
        for k in range(len(tam)):
            ds = [TEN[i] for i in range(len(X)) if nhan[i] == k]
            print(f"  cum {k + 1}: {', '.join(ds)}")
        # Bước dời tâm: trung bình các điểm trong mỗi cụm
        tam_moi = np.array([X[nhan == k].mean(axis=0)
                            for k in range(len(tam))])
        if np.allclose(tam_moi, tam):
            print("  Tam khong doi nua, dung.")
            return tam, nhan
        tam = tam_moi
    return tam, nhan


```

```python
# Khởi tạo cố ý tệ: hai tâm là A và B, nằm sát nhau
tam, nhan = k_means(P, P[[0, 1]].copy())
print()

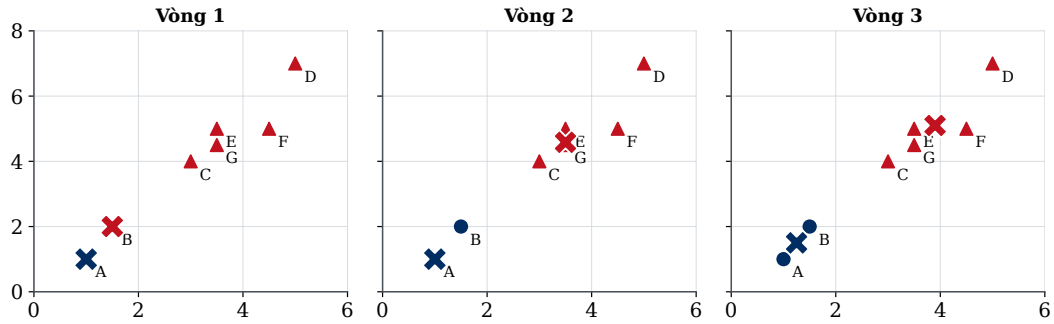
# Tính inertia: tổng bình phương khoảng cách tới tâm cụm của mình
inertia = ((P - tam[nhan]) ** 2).sum()
print(f"Inertia tu tinh: {inertia:.4f}")

# So với scikit-learn, cho cùng tâm ban đầu
km = KMeans(n_clusters=2, init=P[[0, 1]], n_init=1).fit(P)
print(f"Inertia cua scikit-learn: {km.inertia_:.4f}")
print("Tam cua scikit-learn:", np.round(km.cluster_centers_, 2).tolist())
```

```
python code/k1_tu_viet.py
```

```
Vong 1: tam = [[1.0, 1.0], [1.5, 2.0]]
  cum 1: A
  cum 2: B, C, D, E, F, G
Vong 2: tam = [[1.0, 1.0], [3.5, 4.58]]
  cum 1: A, B
  cum 2: C, D, E, F, G
Vong 3: tam = [[1.25, 1.5], [3.9, 5.1]]
  cum 1: A, B
  cum 2: C, D, E, F, G
  Tam khong doi nua, dung.

Inertia tu tinh: 8.5250
Inertia cua scikit-learn: 8.5250
Tam cua scikit-learn: [[1.25, 1.5], [3.9, 5.1]]
```



Hình 1: *Ba vòng của K-Means trên bảy điểm. Dấu X là tâm cụm ở đầu mỗi vòng.*

2.2. Đọc kết quả

- Vòng 1: tâm là A và B. Chỉ A về cụm 1, vì sáu điểm còn lại đều gần B hơn.
- Vòng 2: tâm cụm 2 dời về trung bình của sáu điểm, tức $(3.50, 4.58)$. Giờ B gần A hơn nên chuyển sang cụm 1.
- Vòng 3: tâm dời về $(1.25, 1.50)$ và $(3.90, 5.10)$. Không điểm nào đổi cụm, thuật toán dừng.

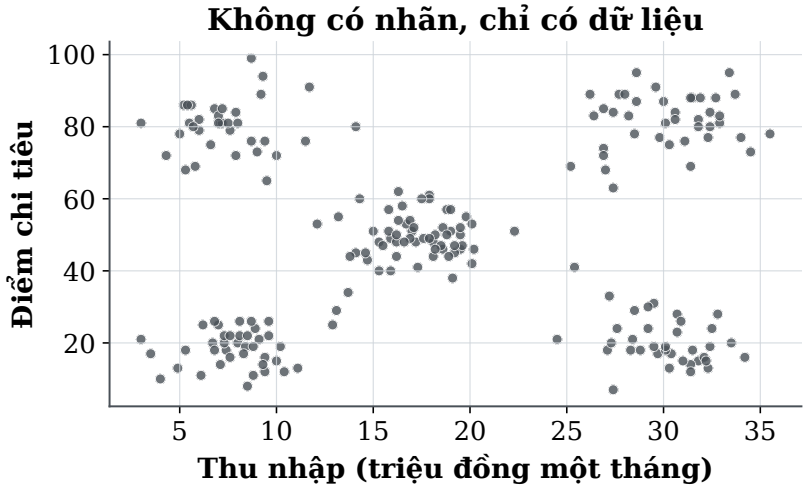
Inertia tự tính bằng 8.5250, trùng với scikit-learn khi cho cùng tâm ban đầu.

Dòng tính khoảng cách

Biểu thức `X[:, None, :] - tam[None, :, :]` tạo ra một mảng ba chiều: với mỗi điểm và mỗi tâm, lấy hiệu tọa độ. Bình phương, cộng theo trục cuối rồi lấy căn là được bảng khoảng cách 7 hàng 2 cột. `argmin(axis=1)` chọn tâm gần nhất cho từng hàng.

3. Phân khúc khách hàng

Bộ phận tiếp thị muốn biết khách hàng chia thành mấy nhóm và mỗi nhóm có đặc điểm gì. Ta phân cụm theo hai cột thu nhập và điểm chi tiêu, rồi dùng cột tuổi để mô tả thêm từng cụm.



Hình 2: *Thu nhập và điểm chi tiêu của 210 khách hàng. Không có nhãn nào.*

```python
# -*- coding: utf-8 -*-
"""Bước 2: phân khúc khách hàng bằng K-Means."""

import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/khach_hang.csv")
print("Kich thuoc bang:", df.shape)
print(df.describe().round(1).to_string())
print()

X = df[["thu_nhap", "diem_chi_tieu"]]
sc = StandardScaler().fit(X)
Xs = sc.transform(X)

km = KMeans(n_clusters=5, n_init=10, random_state=0).fit(Xs)
df["cum"] = km.labels_

# Tâm cụm đang ở thang đã chuẩn hoá, đổi ngược về đơn vị gốc để đọc
```

```
tam = pd.DataFrame(sc.inverse_transform(km.cluster_centers_),
                   columns=["thu_nhap", "diem_chi_tieu"])
tam["so_khach"] = np.bincount(km.labels_)
tam["tuoi_tb"] = df.groupby("cum")["tuoi"].mean()
print("Tam cum, don vi goc:")
print(tam.round(1).to_string())
print()
print(f"Inertia: {km.inertia_:.2f}")
print()

# Khách mới: chuẩn hoá bằng đúng bộ chuẩn hoá cũ rồi tìm tâm gần nhất
moi = pd.DataFrame({"thu_nhap": [29.0, 9.0],
                    "diem_chi_tieu": [85, 75]})
cum_moi = km.predict(sc.transform(moi))
for i in range(len(moi)):
    print(f"Khach moi {i + 1}: thu nhap {moi.iloc[i, 0]},"
          f" chi tieu {moi.iloc[i, 1]} -> cum {cum_moi[i]}")
```

```
python code/k2_khach_hang.py
```

```
Kich thuoc bang: (210, 3)
        tuoi  thu_nhap  diem_chi_tieu
count  210.0     210.0          210.0
mean    37.6      18.5           49.9
std     10.3       9.8           26.8
min     18.0       3.0            7.0
25%     30.0       8.8           22.0
50%     39.0      17.4           49.0
75%     46.0      28.6           77.0
max     62.0      35.5           99.0

Tam cum, don vi goc:
   thu_nhap  diem_chi_tieu  so_khach  tuoi_tb
0       7.5           80.2        35     24.9
1      30.1           20.0        35     42.9
2      17.4           49.3        60     40.4
3       8.0           18.9        40     44.9
4      30.2           81.5        40     32.6

Inertia: 23.02

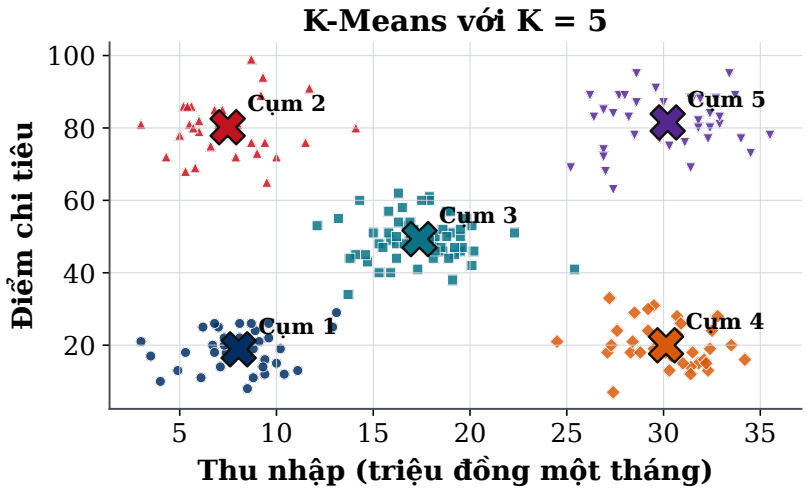
Khach moi 1: thu nhap 29.0, chi tieu 85 -> cum 4
Khach moi 2: thu nhap 9.0, chi tieu 75 -> cum 0
```

3.1. Đọc kết quả

Bảng `describe` cho thấy thu nhập có độ lệch chuẩn 9.8, điểm chi tiêu có độ lệch chuẩn 26.8, hai thang đo khác nhau. Vì vậy phải chuẩn hoá trước khi phân cụm.

Tâm cụm do K-Means trả về nằm trên thang đã chuẩn hoá, khó đọc. Ta đổi chúng về đơn vị

gốc, triệu đồng và điểm, bằng hàm `inverse_transform` của bộ chuẩn hoá `sc`.



Hình 3: *Năm cụm khách hàng. Dấu X là tâm cụm. Số thứ tự cụm trên hình được xếp lại theo thu nhập cho dễ đọc.*

Bảng 3: *Năm cụm khách hàng, theo số thứ tự máy in ra ở trên.*

Cụm	Đặc điểm	Gợi ý cho bộ phận tiếp thị
0	Thu nhập thấp, chi nhiều, trẻ nhất, tuổi trung bình 24.9	Trả góp, chương trình tích điểm
1	Thu nhập cao, chi ít	Tìm hiểu vì sao chưa chi, giới thiệu hàng cao cấp
2	Thu nhập vừa, chi vừa, đông nhất với 60 khách	Giữ chân bằng ưu đãi định kỳ
3	Thu nhập thấp, chi ít	Khuyến mãi giảm giá, hàng thiết yếu
4	Thu nhập cao, chi nhiều	Khách thân thiết, chăm sóc riêng

Số thứ tự cụm không có ý nghĩa

K-Means đánh số cụm từ 0 tới 4 theo thứ tự nó tìm ra, không theo thứ tự nào của dữ liệu. Chạy với `random_state` khác, cụm *thu nhập cao, chi nhiều* có thể mang số 0 thay vì số 4. Luôn đọc tâm cụm để biết cụm nào là cụm nào, đừng nhớ theo số.

3.2. Xếp khách mới vào cụm

Khách mới phải được chuẩn hoá bằng **đúng bộ chuẩn hoá cũ** `sc`, rồi gọi `km.predict` để tìm tâm gần nhất. Khách thu nhập 29 triệu, chi tiêu 85 vào cụm 4, nhóm thu nhập cao chi nhiều. Khách thu nhập 9 triệu, chi tiêu 75 vào cụm 0, nhóm thu nhập thấp chi nhiều.

4. Chọn số cụm K

Mục 3 dùng $K = 5$ vì nhìn hình thấy năm nhóm. Với dữ liệu nhiều cột, không vẽ ra được, ta cần cách chọn K bằng con số.

Hai công cụ chọn K

Khuỷu tay: vẽ inertia theo K, chọn chỗ đường gập, tức chỗ thêm một cụm chỉ giảm inertia rất ít.
Silhouette: với mỗi điểm, gọi a là khoảng cách trung bình tới các điểm cùng cụm, b là khoảng cách trung bình tới các điểm của cụm gần nhất khác.

$$s = \frac{b - a}{\max(a, b)}$$

Lấy trung bình s của mọi điểm, chọn K cho giá trị lớn nhất.

```python
# -*- coding: utf-8 -*-
"""Bước 3: chọn số cụm K bằng khuỷu tay và chỉ số silhouette."""

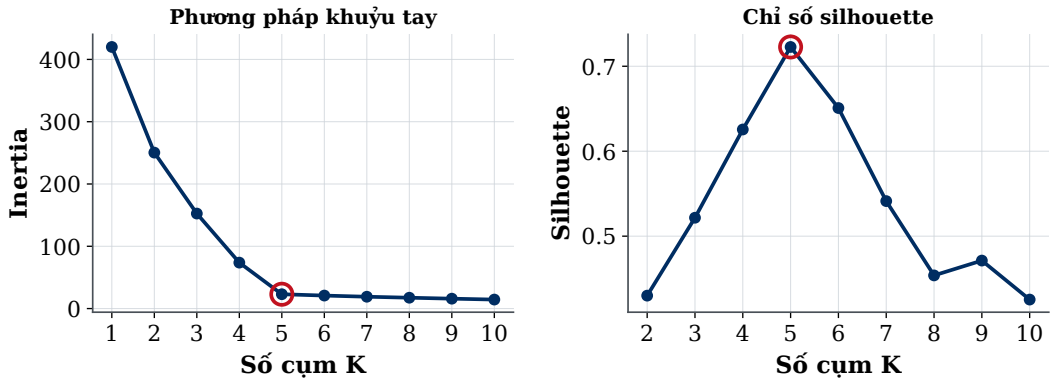
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/khach_hang.csv")
Xs = StandardScaler().fit_transform(df[["thu_nhap", "diem_chi_tieu"]])

print("  K    inertia   giam so voi K-1   silhouette")
truoc = None
for k in range(1, 11):
    km = KMeans(n_clusters=k, n_init=10, random_state=0).fit(Xs)
    giam = "" if truoc is None else f"{truoc - km.inertia_:8.1f}"
    # Silhouette cần ít nhất hai cụm
    sil = "" if k == 1 else f"{silhouette_score(Xs, km.labels_):.3f}"
    print(f"  {k:2d}   {km.inertia_:7.1f}   {giam:>15s}   {sil:>10s}")
    truoc = km.inertia_
```

```
python code/k3_chon_k.py
```

```
 K    inertia   giam so voi K-1   silhouette
 1      420.0
 2      250.4            169.6        0.430
 3      152.5             97.9        0.522
 4       73.8             78.8        0.626
 5       23.0             50.7        0.723
 6       20.9              2.1        0.651
 7       19.0              1.9        0.541
 8       17.4              1.6        0.454
 9       15.9              1.5        0.471
10       14.4              1.4        0.425
```



Hình 4: *Inertia và chỉ số silhouette theo số cụm K. Vòng tròn đỏ đánh dấu $K = 5$.*

4.1. Đọc kết quả

Cột *giảm so với K-1* là chỗ dễ đọc nhất. Từ $K = 4$ lên $K = 5$, inertia còn giảm được 50.7. Từ $K = 5$ lên $K = 6$, inertia chỉ giảm 2.1, và từ đó trở đi mỗi cụm thêm vào chỉ giảm chưa tới 2. Đường gập ở $K = 5$.

Cột silhouette cao nhất ở $K = 5$, bằng 0.723, rồi giảm dần khi K tăng. Hai công cụ cùng chỉ ra $K = 5$.

Không chọn K cho inertia nhỏ nhất

Inertia luôn giảm khi K tăng. Với K bằng số khách, mỗi khách là một cụm và inertia bằng 0, nhưng ta chẳng tìm ra nhóm nào.

Dữ liệu thật hiếm khi rõ như vậy

Bộ dữ liệu này được mô phỏng với năm nhóm tách bạch, nên hai công cụ đồng ý với nhau. Với dữ liệu thật, đường khuỷu tay thường cong đều và silhouette có vài đỉnh gần bằng nhau. Khi đó số cụm còn phải dựa vào mục đích sử dụng, ví dụ bộ phận tiếp thị đủ sức chạy mấy chương trình.

5. Ba cái bẫy của K-Means

```python
# -*- coding: utf-8 -*-
"""Bước 4: ba cái bẫy của K-Means."""

import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/khach_hang.csv")
X = df[["thu_nhap", "diem_chi_tieu"]]
Xs = StandardScaler().fit_transform(X)

# Bẫy 1: thu nhập ghi bằng nghìn đồng, không chuẩn hoá
X_nghin = X.copy()
```

```python
X_nghin["thu_nhap"] = X_nghin["thu_nhap"] * 1000
km_tho = KMeans(n_clusters=5, n_init=10, random_state=0).fit(X_nghin)
tam_tho = km_tho.cluster_centers_.copy()
tam_tho[:, 0] = tam_tho[:, 0] / 1000         # đổi lại triệu đồng để dễ đọc
print("Bay 1: khong chuan hoa, tam cum (thu nhap trieu, chi tieu):")
for t in tam_tho[np.argsort(tam_tho[:, 0])]:
    print(f"  ({t[0]:5.1f}, {t[1]:5.1f})")
print()

# Bẫy 2: mỗi lần chỉ khởi tạo ngẫu nhiên một lần
ket = []
for rs in range(20):
    km = KMeans(n_clusters=5, init="random", n_init=1,
                random_state=rs).fit(Xs)
    ket.append(round(km.inertia_, 1))
print("Bay 2: inertia cua 20 lan khoi tao ngau nhien:")
ket = sorted(ket)
print(" ", ket[:10])
print(" ", ket[10:])
tot = KMeans(n_clusters=5, n_init=10, random_state=0).fit(Xs)
print(f"  Voi n_init=10: inertia {tot.inertia_:.1f}")
print()

# Bẫy 3: cụm không tròn, dữ liệu hai vầng trăng
tr = pd.read_csv("data/hai_vang_trang.csv")
km_tr = KMeans(n_clusters=2, n_init=10, random_state=0)
nhan = km_tr.fit_predict(tr[["x1", "x2"]])
khop = (nhan == tr["nhan"]).mean()
# Số thứ tự cụm có thể ngược với nhãn thật, nên lấy cách khớp tốt hơn
print(f"Bay 3: hai vang trang, ty le khop nhan that "
      f"{max(khop, 1 - khop):.4f}")
```

```
python code/k4_ba_cai_bay.py
```

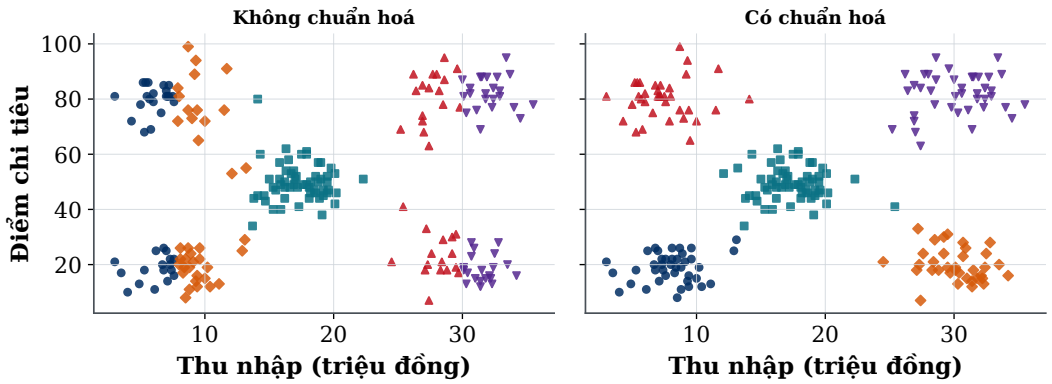
```
Bay 1: khong chuan hoa, tam cum (thu nhap trieu, chi tieu):
  (  6.1,  52.5)
  (  9.6,  41.9)
  ( 17.4,  49.8)
  ( 27.8,  52.9)
  ( 31.9,  52.4)

Bay 2: inertia cua 20 lan khoi tao ngau nhien:
  [23.0, 23.0, 23.0, 23.0, 23.0, 23.0, 23.0, 23.0, 23.0, 23.0]
  [72.4, 73.6, 73.7, 73.7, 84.0, 84.2, 84.2, 116.6, 117.4, 118.2]
  Voi n_init=10: inertia 23.0

Bay 3: hai vang trang, ty le khop nhan that 0.7500
```

5.1. Bẫy thứ nhất: quên chuẩn hoá

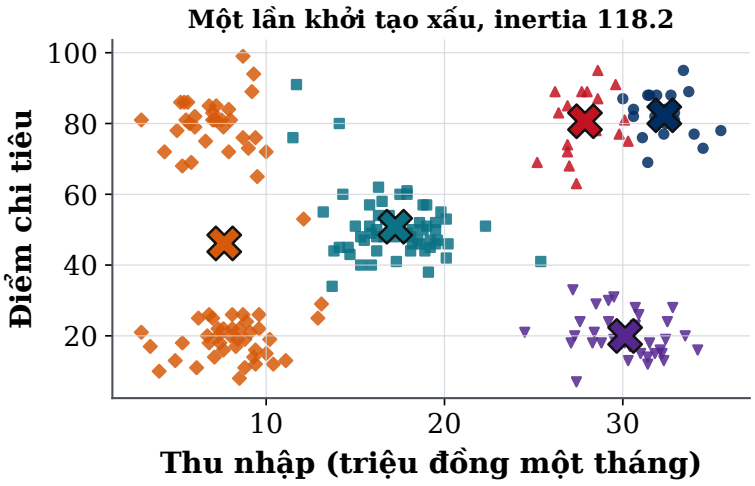
Khi thu nhập ghi bằng nghìn đồng và không chuẩn hoá, năm tâm cụm có điểm chi tiêu gần như bằng nhau, từ 41.9 tới 52.9. Chúng chỉ khác nhau ở thu nhập. Nghĩa là K-Means đã chia khách thành năm **dải dọc** theo thu nhập, bỏ qua hẳn điểm chi tiêu.



Hình 5: *Bên trái: thu nhập tính bằng nghìn đồng, không chuẩn hoá. Bên phải: có chuẩn hoá.*

5.2. Bẫy thứ hai: khởi tạo xấu

Hai mươi lần chạy, mỗi lần chỉ khởi tạo ngẫu nhiên một lần, in ra theo thứ tự tăng dần: 10 lần đạt inertia 23.0, 10 lần còn lại dừng ở những cách chia kém hơn, tệ nhất tới 118.2.

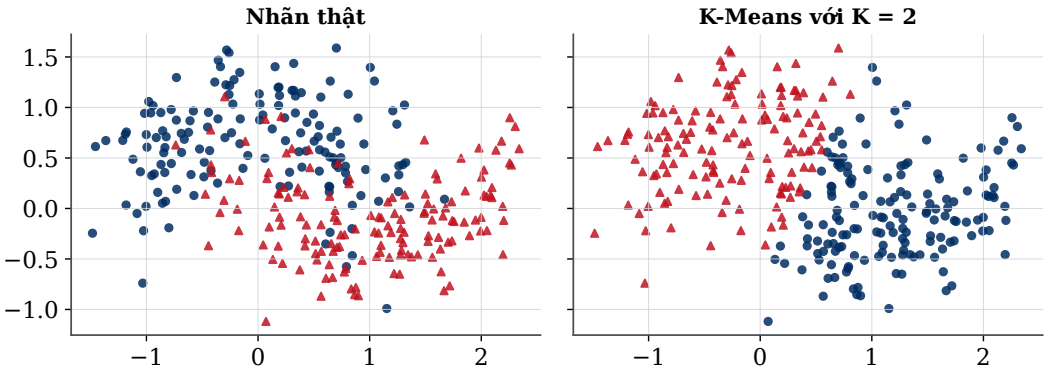


Hình 6: *Lần khởi tạo tệ nhất: hai nhóm thu nhập thấp bị gộp làm một, còn nhóm thu nhập cao chi nhiều bị cắt đôi.*

K-Means chỉ dừng khi không bước nào cải thiện được nữa, chứ không đảm bảo tìm ra cách chia tốt nhất. Với `n_init=10`, thư viện chạy 10 lần và giữ lần tốt nhất, nên được 23.0. Thêm vào đó, cách khởi tạo mặc định `"k-means++"` chọn các tâm ban đầu nằm xa nhau, ít khi rơi vào cách chia xấu.

5.3. Bẫy thứ ba: cụm không tròn

Trên hai vầng trăng, cách chia của K-Means chỉ khớp nhãn thật 0.7500.



Hình 7: *Bên trái là nhãn thật. Bên phải là hai cụm của K-Means, bị cắt gần như bằng một đường thẳng.*

Mỗi điểm về tâm gần nhất, nên ranh giới giữa hai cụm luôn là đường trung trực của hai tâm, tức là một đường thẳng. K-Means không thể đi theo hình cong của hai vầng trăng.

Tỷ lệ khớp chỉ dùng được khi có nhãn

Ở đây ta có nhãn thật nên mới đo được tỷ lệ khớp. Mã lấy `max(khop, 1 - khop)` vì cụm số 0 của K-Means có thể ứng với nhãn 1, số thứ tự cụm không có ý nghĩa.

6. Nén màu ảnh

Một bức ảnh màu là một bảng các điểm ảnh, mỗi điểm có ba con số đỏ, xanh lá, xanh dương, từ 0 tới 255. Mỗi điểm ảnh là một điểm trong không gian ba chiều. Phân cụm các điểm này thành K cụm, rồi tô mỗi điểm bằng màu của tâm cụm, ta được một ảnh chỉ còn K màu.

```
# -*- coding: utf-8 -*-
"""Bước 5: nén màu một bức ảnh bằng K-Means."""

import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import load_sample_image

anh = load_sample_image("china.jpg")        # ảnh có sẵn trong scikit-learn
cao, rong, _ = anh.shape
diem = anh.reshape(-1, 3) / 255.0           # mỗi dòng là một điểm ảnh RGB
so_mau = len(np.unique(anh.reshape(-1, 3), axis=0))
print(f"Anh {cao} x {rong}, co {cao * rong} diem anh, {so_mau} mau")
print()

# Học trên 5000 điểm ảnh rút ngẫu nhiên cho nhanh, rồi gán cho mọi điểm
rng = np.random.default_rng(0)
mau = diem[rng.choice(len(diem), 5000, replace=False)]

print("  K   so bit moi diem   nho hon   sai lech trung binh")
for k in [4, 16, 64]:
    km = KMeans(n_clusters=k, n_init=4, random_state=0).fit(mau)
    nhan = km.predict(diem)
    moi = km.cluster_centers_[nhan]
```

```
    bit = int(np.ceil(np.log2(k)))
    sai = np.abs(moi - diem).mean() * 255
    print(f"  {k:2d}        {bit}            {24 / bit:4.1f} lan"
          f"      {sai:5.2f} muc sang")
```

```
python code/k5_nen_anh.py
```

```
Anh 427 x 640, co 273280 diem anh, 96615 mau

  K   so bit moi diem   nho hon   sai lech trung binh
  4         2            12.0 lan      17.41 muc sang
 16         4             6.0 lan       7.97 muc sang
 64         6             4.0 lan       4.54 muc sang
```



Hình 8: *Ảnh gốc, ảnh chỉ còn 16 màu và ảnh chỉ còn 4 màu.*

6.1. Đọc kết quả

Ảnh gốc có 273280 điểm ảnh dùng 96615 màu khác nhau. Mỗi điểm lưu đủ ba số 8 bit, tức 24 bit.

Với 16 màu, mỗi điểm chỉ cần lưu số thứ tự màu từ 0 tới 15, tức 4 bit, nhỏ hơn 6 lần. Mỗi điểm ảnh lệch trung bình 7.97 mức sáng trên thang 255, mắt thường khó nhận ra như Hình 8. Với 4 màu, nhỏ hơn 12 lần nhưng lệch tới 17.41 mức sáng, bầu trời và cây cối bị loang rõ.

Vì sao chỉ học trên 5000 điểm

Chạy K-Means trên cả 273280 điểm khá chậm. Tâm cụm học từ 5000 điểm rút ngẫu nhiên đã đủ đại diện cho các màu của ảnh. Học xong, `km.predict` gán màu cho mọi điểm ảnh rất nhanh.

7. Bài tập

Sinh viên tạo thư mục `baitap08` nằm ngay trong thư mục bài thực hành. Vẫn chạy lệnh từ thư mục gốc, tức gõ `python baitap08/bai1.py`, và trong mã vẫn viết `data/...`. Bài nộp gồm tệp mã nguồn và ảnh chụp màn hình phần kết quả.

Bài tập 1: Tự tính silhouette

Viết hàm `silhouette_mot_diem(X, nhan, i)` tính a, b và s của điểm thứ i theo đúng định nghĩa ở mục 4, chỉ dùng numpy. Chạy hàm cho mọi khách hàng với $K = 5$, lấy trung bình,

rồi so với `silhouette_score`.
Khách nào có s nhỏ nhất? Khách đó nằm ở đâu trên Hình 3?

Bài tập 2: Thêm cột tuổi

Phân cụm trên cả ba cột tuổi, thu nhập và điểm chi tiêu, có chuẩn hoá. Chọn lại K bằng silhouette với K từ 2 tới 10.
K tốt nhất có còn là 5 không? Tâm các cụm thay đổi thế nào so với mục 3?

Bài tập 3: K-Means++ và khởi tạo ngẫu nhiên

Chạy K-Means 50 lần với `n_init=1`, `random_state` từ 0 tới 49, lần lượt với `init="random"` và `init="k-means++"`. Đếm xem mỗi cách có bao nhiêu lần đạt inertia 23.0. Cách khởi tạo nào đáng tin hơn?

Bài tập 4: Lưu ảnh đã nén

Dựa vào `k5_nen_anh.py`, tạo ảnh 16 màu rồi lưu bằng `plt.imsave("anh_16_mau.png", anh_moi)`, với `anh_moi` là mảng có dạng `(427, 640, 3)`. Lưu cả ảnh gốc, rồi so dung lượng hai tệp.
Làm lại với một bức ảnh của chính em, đọc bằng `plt.imread`. Chú ý ảnh PNG có thể có kênh thứ tư là độ trong suốt, phải bỏ kênh đó trước khi phân cụm.

Bài tập 5: DBSCAN trên hai vầng trăng

Dùng `DBSCAN(eps=e, min_samples=5)` trong `sklearn.cluster` trên bộ hai vầng trăng, với `e` lần lượt bằng 0.2, 0.3 và 0.4. Với mỗi giá trị, đếm số cụm tìm được và số điểm bị gán nhãn -1, tức điểm nhiễu.
Có giá trị nào tách được hai vầng trăng tốt hơn K-Means không? Nhận xét vai trò của `eps`.

Bài tập 6: Phân cụm chữ số viết tay

Chạy K-Means với $K = 10$ trên bộ chữ số viết tay `load_digits`, không dùng nhãn. Sau đó mới dùng nhãn thật: lập bảng `pd.crosstab(nhan_that, cum)` để xem mỗi cụm chứa những chữ số nào.
Chữ số nào được gom gọn vào một cụm? Những chữ số nào hay bị gom chung?

Tiêu chí chấm

Mã chạy được không lỗi, kết quả in ra đúng, có nhận xét bằng lời ở những bài yêu cầu, tên biến đặt có nghĩa. Bài 1 và bài 3 chấm nặng phần giải thích, vì đó là chỗ kiểm tra sinh viên có hiểu silhouette và vai trò của khởi tạo hay không.

8. Phụ lục A, bảng tra nhanh

Bảng 4: *Các hàm mới xuất hiện trong bài 8.*

Lệnh	Thư viện	Công dụng
`KMeans`	scikit-learn	Thuật toán phân cụm K-Means.
`n_clusters`	scikit-learn	Số cụm K.
`n_init`	scikit-learn	Số lần chạy với tâm ban đầu khác nhau, giữ lần tốt nhất.
`init`	scikit-learn	Cách chọn tâm ban đầu, mặc định `"k-means++"`.
`.labels_`	scikit-learn	Số thứ tự cụm của từng mẫu.
`.cluster_centers_`	scikit-learn	Tọa độ các tâm cụm.
`.inertia_`	scikit-learn	Tổng bình phương khoảng cách tới tâm cụm.
`.predict`	scikit-learn	Xếp mẫu mới vào cụm có tâm gần nhất.
`silhouette_score`	scikit-learn	Chỉ số silhouette trung bình của cả cách chia.
`inverse_transform`	scikit-learn	Đổi dữ liệu đã chuẩn hoá về đơn vị gốc.
`load_sample_image`	scikit-learn	Nạp ảnh mẫu có sẵn trong thư viện.
`np.bincount`	numpy	Đếm số lần xuất hiện của mỗi số nguyên.

Bảng 5: *Các con số kết quả chính của bài thực hành.*

Đại lượng	Giá trị
Bảy điểm, số vòng lặp tới khi dừng	3
Bảy điểm, inertia sau cùng	8.5250
Khách hàng, inertia với $K = 5$	23.0
Silhouette với $K = 4, 5, 6$	0.626, 0.723, 0.651
Khởi tạo ngẫu nhiên một lần, số lần tệ trên 20	10
Hai vầng trăng, tỷ lệ khớp nhãn thật	0.7500
Ảnh 16 màu, nhỏ hơn bao nhiêu lần	6.0
Ảnh 16 màu, sai lệch trung bình	7.97 mức sáng

9. Phụ lục B, lỗi thường gặp

Bảng 6: *Các lỗi hay gặp riêng của bài phân cụm.*

Thông báo lỗi hoặc hiện tượng	Nguyên nhân và cách xử lý
Các cụm thành những dải song song	Quên chuẩn hoá, một cột có thang đo lớn nuốt trọn khoảng cách. Dùng `StandardScaler` trước khi phân cụm.
Mỗi lần chạy ra một kết quả	Quên `random_state`, hoặc đặt `n_init=1`.
Tâm cụm có giá trị lạ như -0.8 hay 1.2	Đang đọc tâm trên thang đã chuẩn hoá. Dùng `sc.inverse_transform` để đổi về đơn vị gốc.
`ValueError: Number of labels is 1`	Gọi `silhouette_score` với $K = 1$. Silhouette cần ít nhất hai cụm.
Khách mới bị xếp vào cụm vô lý	Quên chuẩn hoá khách mới, hoặc chuẩn hoá bằng một `StandardScaler` khác. Phải dùng đúng bộ chuẩn hoá đã học.
Chạy rất lâu trên ảnh lớn	Học trên toàn bộ điểm ảnh. Học trên vài nghìn điểm rút ngẫu nhiên rồi dùng `predict`.

Một lời khuyên chung

Với K-Means, luôn chuẩn hoá trước, luôn đặt `n_init` và `random_state`, và luôn đọc tâm cụm theo đơn vị gốc. Máy chỉ chia cụm, việc hiểu và đặt tên cho từng cụm là của con người.